让你的大模型读懂二方包

原创 用户互动团队 大淘宝技术 2025年9月8日 02:46 浙江 14人

本文主要讲述了在开发过程中，大模型（如AI编程助手）难以理解项目所依赖的二方包代码的问题，并通过一个实际案例复盘了从“翻车”到“驯服”AI的全过程。作者尝试使用Cursor等AI工具直接生成调用二方包接口的代码，但因AI无法读取未在当前工程中打开的依赖源码，导致生成的代码错误频出，甚至出现幻觉式编码。为解决此问题，作者探索了多种方案，最终发现，本地反编译MCP方案最有效，能精准解析二方包中的类与方法，显著提升代码生成的准确性和可用性。文章强调，应将AI视为需辅助的“工具人”，通过提供良好上下文与工具来增强其能力，而非期待其无所不能。

01 背景

近期有一个需求中需要查询用户订单状态，这个功能本身很简单，是一个经典的依赖下游场景，需要读取交易订单二方包的代码逻辑，并分析出入参出参的含义，并判定用户指定的订单是否已经进入终态。

在我们大多数日常工作开发过程中，遇到的很多需要调用各种下游提供的接口，对二方包有依赖的场景（例如优惠券、会员、订单……）这种情况下，大模型是否能够读懂我们的诉求呢?

本来以为这么简单的需求，这些ai还不是手拿把掐，结果却上演了一出大型“翻车现场”。这篇文章，就是我从“翻车”到“驯服”的全过程复盘，告诉你如何让大模型读懂二方包。本文对部分方法名称做了脱敏，可能会有一些方法名称前后不一致，不影响核心论述点。

02 直接使用cursor — 失败

一开始，我信心满满。我把交易接口的内部文档链接喂给了Cursor，并用**/doc功能添加了交易那边的两个相关接口页：查询订单（在线库）、查询订单（历史库）**（这两个网页是内部办

公网的网页，大概率外部是打不开的），不过先试试吧？我清晰地描述了我的需求：“请帮我实现一个批量查询订单的方法，先查在线库，查不到的再去历史库捞……”下面是几轮的对话记录：

user：

请帮我实现一个批量查询订单的方法，查询指定userId和List<long> subOrderIdList下的订单结果。请在TpClientImpl中实现。请注意方法的逻辑。需要先在通过订单在线库中查询，使用`xxxService.queryMainAndDetail`，如果查询出来的结果中没有指定orderIdList的信息，请收集之后统一再次使用订单历史库查询，使用`xxxBaseService.queryMainAndDetail`。根据网页中给出的返回值的逻辑。最终将两者查询出来的结果统一按照传入的subOrderIdList的顺序返回一个订单列表。请先不要写代码，先规划任务并写入到.ai/指定订单查询task.md。

Cursor：

给出了需要实现一个批量查询订单的方法 `queryBizOrderList`，下面是任务规划文档，写的没啥问题：

```
# 指定订单查询任务规划

## 需求概述
实现批量查询指定 userId 和 List<long> subOrderIdList 下订单结果的方法 `query

## 业务逻辑
1. **先查询在线库**：使用 `xxxService.queryMainAndDetail` 方法查询传入的所有
2. **收集未找到的订单ID**：对比查询结果与传入的订单ID列表，收集未查询到的订单ID
3. **查询历史库补充**：使用 `xxxBaseService.queryMainAndDetail` 方法查询未找
4. **合并结果**：将在线库和历史库的查询结果合并
5. **按序返回**：按照传入的 List<long> subOrderIdList 的顺序重新排序并返回订单

## 技术实现任务分解

### 任务1：了解现有服务和方法签名
- [ ] 确认 `xxxService.queryMainAndDetail` 方法是否存在，如不存在需要定义
- [ ] 确认 `xxxBaseService`（历史库服务）是否存在，如不存在需要引入
- [ ] 确认 `xxxBaseService.queryMainAndDetail` 方法签名
- [ ] 了解查询结果的数据结构和返回值类型

### 任务2：设计方法签名
```java
/**
 * 批量查询指定用户的订单列表
 * @param userId 用户ID
 * @param subOrderIdList 子订单ID列表
 * @return 按照传入顺序返回的订单结果列表
```
```

```
 * @throws TCException 查询异常
 */
List<SingleQueryResultDO> queryBizOrderList(long userId, List<Long> su
```

任务3：实现核心逻辑
- [ ] 参数校验：检查 userId 和 subOrderIdList 的有效性
- [ ] 在线库批量查询：调用 `xxxService.queryMainAndDetail`
- [ ] 结果分析：识别哪些订单ID没有查询到结果
- [ ] 历史库补充查询：对未找到的订单ID调用 `xxxBaseService.queryMainAndDetai
- [ ] 结果合并：将两次查询的结果合并到一个Map中
- [ ] 顺序重排：按照传入的 subOrderIdList 顺序构建返回列表

任务4：异常处理和日志
- [ ] 添加适当的异常处理逻辑
- [ ] 对查询不到的订单ID记录ERROR日志（根据记忆中的用户偏好）
- [ ] 使用 guard clauses（提前返回）优化代码结构
- [ ] 添加 TraceLogUtil 链路追踪
- [ ] 统计在线库和历史库的查询耗时

任务5：性能优化考虑
- [ ] 如果传入的订单ID列表为空，提前返回
- [ ] 避免不必要的历史库查询（如果在线库已查询到所有订单）
- [ ] 考虑是否需要并行查询在线库和历史库

任务6：更新接口定义
- [ ] 在 xxxClient 接口中添加 `queryBizOrderList` 方法定义
- [ ] 确保方法签名的一致性

任务7：单元测试
- [ ] 编写测试用例
- [ ] 测试场景：全部在在线库找到
- [ ] 测试场景：部分在在线库，部分在历史库
- [ ] 测试场景：全部需要从历史库查询
- [ ] 测试场景：部分订单查询不到
- [ ] 测试场景：异常处理

依赖和风险点

需要确认的依赖
1. **xxxBaseService 服务**：需要确认历史库查询服务是否已配置
2. **queryMainAndDetail 方法**：需要确认具体的方法签名和返回值结构
3. **数据一致性**：确认在线库和历史库返回的数据结构是否一致

潜在风险
1. **性能风险**：批量查询可能影响性能，需要考虑分页或限制查询数量

```
2. **超时风险**：历史库查询可能较慢，需要设置合适的超时时间
3. **数据不一致**：在线库和历史库的数据可能存在不一致

## 实现优先级
1. 高优先级：核心查询逻辑实现
2. 中优先级：异常处理和日志记录
3. 低优先级：性能优化和监控
```

并且让我确认依赖关系：

- `xxxBaseService` 服务需要引入；
- `queryMainAndDetail` 方法签名需要确认；
- 批量查询的性能和超时控制。

这里做的不错。

user：

请先阅读二方包代码 `xxxService.queryMainAndDetail` 以及 `xxxBaseService.queryMainAndDetail` 调用的出入参是什么，再写代码逻辑。

Cursor：

其阅读了代码后给了我他的判定结果：

1. `xxxService.queryMainAndDetail` 方法不存在；
2. `xxxBaseService` 服务不存在，
   代码库中没有找到历史库查询服务 `xxxBaseService` ；
3. `queryMainAndDetail` 方法在整个代码库中都不存在。

走到这里判定结果，其实就说明了Cursor只读取了我们工程代码库的代码，并不能读取我跟他明确指定的二方包中的方法。另外因为之前在我们写过查询单个指定id订单的方法，Cursor提出基于现有的方法实现批量查询：使用循环调用 `getBizOrderById` 方法实现批量查询。

这里可以看出Cursor并不能读取二方包中代码，我以为是因为我没说清楚，所以下一轮对话中我明确说了全限定名。

user：

这两个类以及方法是二方包中的方法，二方包我已经在pom中以及Configuration引入了，需要你阅读并使用com.tb.xxx.yyy.api.xxxService和com.tb.ccc.ddd.service.xxxBaseService，调用前面我提到的`queryMainAndDetail` 方法。

Cursor：

开始随意实现了。。。导入的包路径是LLM根据代码库中之前client防腐层中写过的getBizOrderById方法中引入的xxxService幻想出来的。
com.tb.xxx.yyy.api.xxxBaseService，并没有用正确的com.tb.ccc.ddd.service.xxxBaseService。
Cursor发现报错之后呢，再次进行猜测转成了com.tb.xxx.yyy.api.history.xxxBaseService....😅 就是纯根据我跟他说这是一个历史库查询，自己在幻觉了（假装自己把活完成的很好）。。。

这里我就介入了，手动给他修改了import，但是我并没有直接告诉大模型xxxService.queryMainAndDetail以及xxxBaseService.queryMainAndDetail 的完整方法签名（残存着希望对ai的幻想）。

user：
好了，我已经修正了引入了xxxService和txxxBaseService的导入路径，请你解析我指定的方法，根据出入参写出正确的调用代码。

Cursor：
cursor 并没有直接读取 xxxService.queryMainAndDetail 以及 xxxBaseService.queryMainAndDetail 的方法签名，而是根据编译后返回的错误信息才知道其传入的参数不正确，下面是其思考过程：

```
好的，现在我看到了方法签名的错误。需要使用 QueryBizOrderDO 对象而不是直接传入 List
我需要了解 QueryBizOrderDO 的结构。让我搜索更多相关信息：
让我继续搜索 QueryBizOrderDO 的相关信息：
看起来我无法在代码库中找到 QueryBizOrderDO 的具体实现。
```

Cursor发现其根本不知道应该如何构造这个入参。其实进行到这里已经非常确认Cursor无法读取到jar包中的代码了（即便我已经说了全限定名的情况下）。

进行到这里，我感觉自己不是在和AI结对编程，而是在辅导一个记性不好还爱幻想的孩子写作业。另外看了一下入参 xxxQueryXxxOrderDO 定义有两千多行，返回对象 xxxOrderInfoResultDO 内部也有多层逻辑。



参数名称	参数类型	参数描述
[redacted]	[redacted]	
• totalNumber	int	订单总数
• orderList	List<[redacted]>	
◦ bizType	int	业务类型
◦ bizOrderDO	[redacted]	订单表，包含支付状态、物流状态、退款状态、商品标题、订单标等信息
[redacted]	[redacted]	物流表，包括收货地址、手机号等；
[redacted]	[redacted]	支付表
[redacted]	[redacted]	退款表

其中我要用到的订单信息 bizOrderDO 又有上万行。我要是把这些代码都粘给ai我估计上下文token要爆炸，为了保护我的额度和我这个月的（摸鱼）幸福，还是算了吧。

另外理论上可以考虑直接把这个api的网页导出到markdown形式试一试，只不过我看了网页里面还嵌套了语雀文档，语雀文档中再去叙述出参里面代表了什么，而且网页里面的信息我看了一下和实际二方包也不符，比如要传入的订单id，网页中显示是long，实际二方包是传入的long[]，所以就放弃了。



• bizOrderId	long		业务订单ID，为单条查询提供

【世上无难事，只要肯放弃（不是）】

对于这个需求，无往而不利的ai像“智障”一样，我意识到一个问题：对于未在当前工作区打开的、作为编译依赖（二方包）存在的代码，LLM几乎是“盲人”。它无法像IDE那样“Go to Definition”，也无法理解那些复杂对象的内部结构。指望通过喂文档链接来解决，在面对复杂的内部系统时，基本等于缘木求鱼。

痛定思痛，我命由我不由天！既然它看不见，我就想办法让它“开天眼”。我大概调研使用了以下几个方案。

03 我命由我不由天–让大模型读懂你

当前的AI工具本质上无法理解二方包的说明，这些ai工具会尝试着欺骗你，会企图用一些自己拼凑内容假装自己干好了你给布置给他的活，而且不仔细看的话甚至觉得他完成的还可以。正

如业务会通过数据欺骗老板的眼睛营造出一种欣欣向荣的假象，正如你会跟老板说这个活我干完了不管用了什么奇怪的方式，反正最终活我肯定是干完了的！🐶

前面说了对于这种需要依赖下游二方包的需求，llms需要拿到外部依赖的知识来生成代码，那我们就需要提供工具给他让他了解这部分知识。类比我们如果是一个刚接手这个代码仓库的新同学，我们是如何熟悉项目的呢，大概率是看项目知识库，以及在idea内各种跳转看源码。

对于大模型也是一样的，也有两种方案，对比如下：

方案	优点	缺点
构建项目知识库：维护当前项目所需要的内容以及外部的依赖说明，对于外部的依赖说明可以将接口类、出入参类都拷贝进去。	1.能够让LLM对当前项目有一个全面的理解。 2.人可以加入理解说明，相关的一些参数在接口上说明并不全面，需要人维度的解读。	1.建立的知识库对当前项目的理解比较深入但是对于外部的依赖理解还是局限于自身的使用上。 2.全集团维护一个公共的完整知识库才能很好地解决外部依赖问题。 3.知识召回的准确度问题，可能会召回不需要的的内容。
实现类idea的索引查询：开发的时候能够像人一样点击到定义处了解到具体的类信息。	1.召回的类数据一定是最准确的。 2.可以召回类依赖数据，确保LLM对某个类的理解是全面而完整的。	1.知识强依赖于类的说明和注解，错误的注解会影响最终效果。 2.包需要下载到本地.m2，分析可能token耗用量比较大。

如何让这些AI工具拿到二方包的内容呢进而实现需求呢，我大概调研了以下几种方案。

方案一 & 二：大力出奇迹 — 手动“投喂”源码

最直接的思路，就是缺啥补啥。既然LLM看不到二方包的源码，那我就把源码复制给它不就行了？这显然是最无脑的方案，这里先不赘述手动将源码复制到聊天框的方案了。

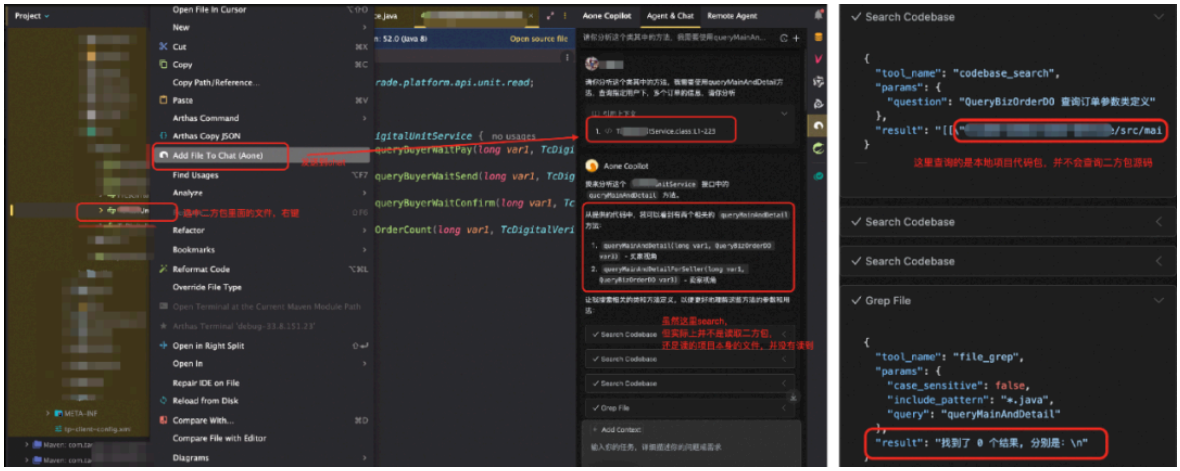
我分别尝试了公司内部的AoneCopilot和通用的Claude，可以看看他们提供了一些读取二方包代码的方案，不需要我们手动粘贴到对话框。

- 方案一：使用AoneCopilot

1. 找到选定文件后右键选择直接发送文件到chat

可以看到AoneCopilot能够分析指定文件的代码信息，可以看到其分析二方包的方法，其入参是QueryBizOrderDO，但是因为我并没有手动将二方包中的QueryBizOrderDO类add file to chat，**agent并不会在二方包中找这个DTO**，还是在项目代码中读取，并没有找到入参的定义。

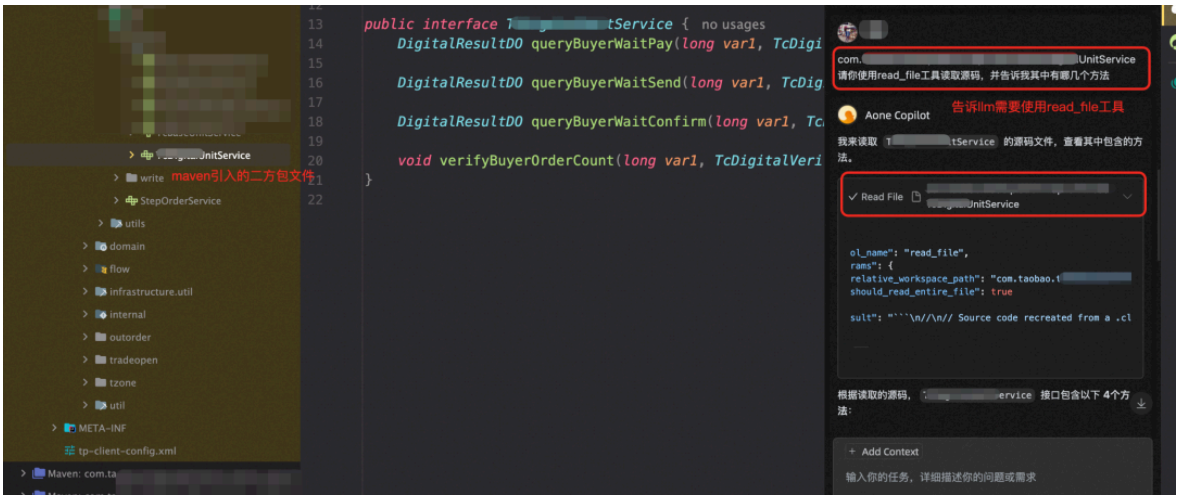
这个方案其实跟手动粘贴代码给agent并没有什么区别，只能读取手动添加进去的文件，对于没手动添加到chat的文件agent是不能读到的。



你可能会有疑问，如果这个二方包没有上传源码的情况，agent能否拿到代码内容呢？我试了一下，发现是可以拿到代码呢，我在AoneCopilot文档中看了一下他们回答：当使用“Add File To Chat”功能时，IDE已经帮您完成了文件读取和反编译工作，并将结果直接提供给AoneCopilot，所以此时他能够直接拿到反编译的代码内容。

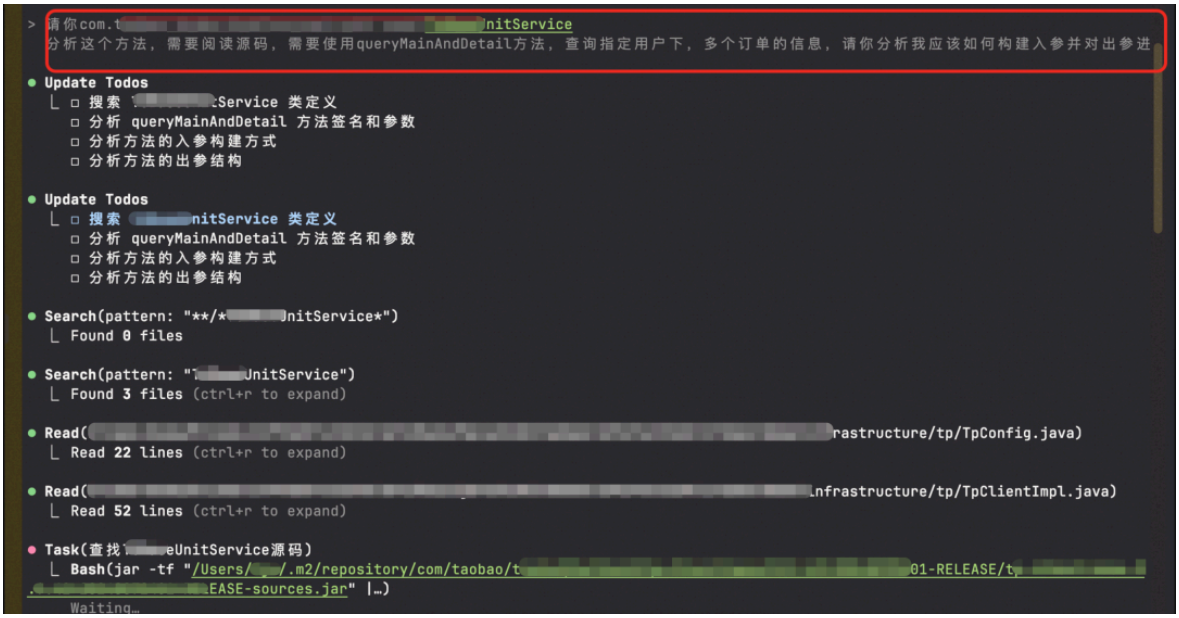
2、告知使用read_file工具

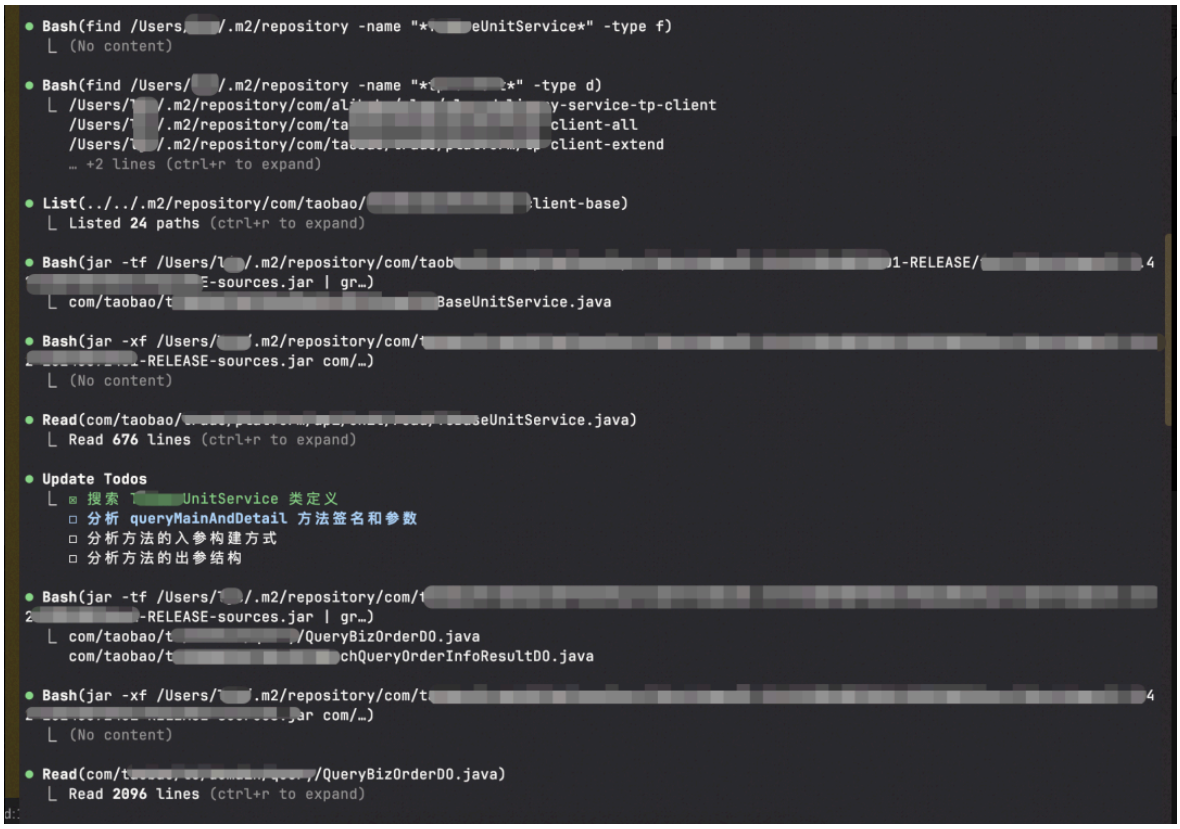
可以指定全限定名，并告知其使用read_file工具读取代码，这个方案可以让AoneCopilot分析这个类的代码信息，并且也能够分析我没有指定的入参的类的信息。这个方案看起来已经比较智能了，但是貌似只能读取有源码的二方包中的内容。

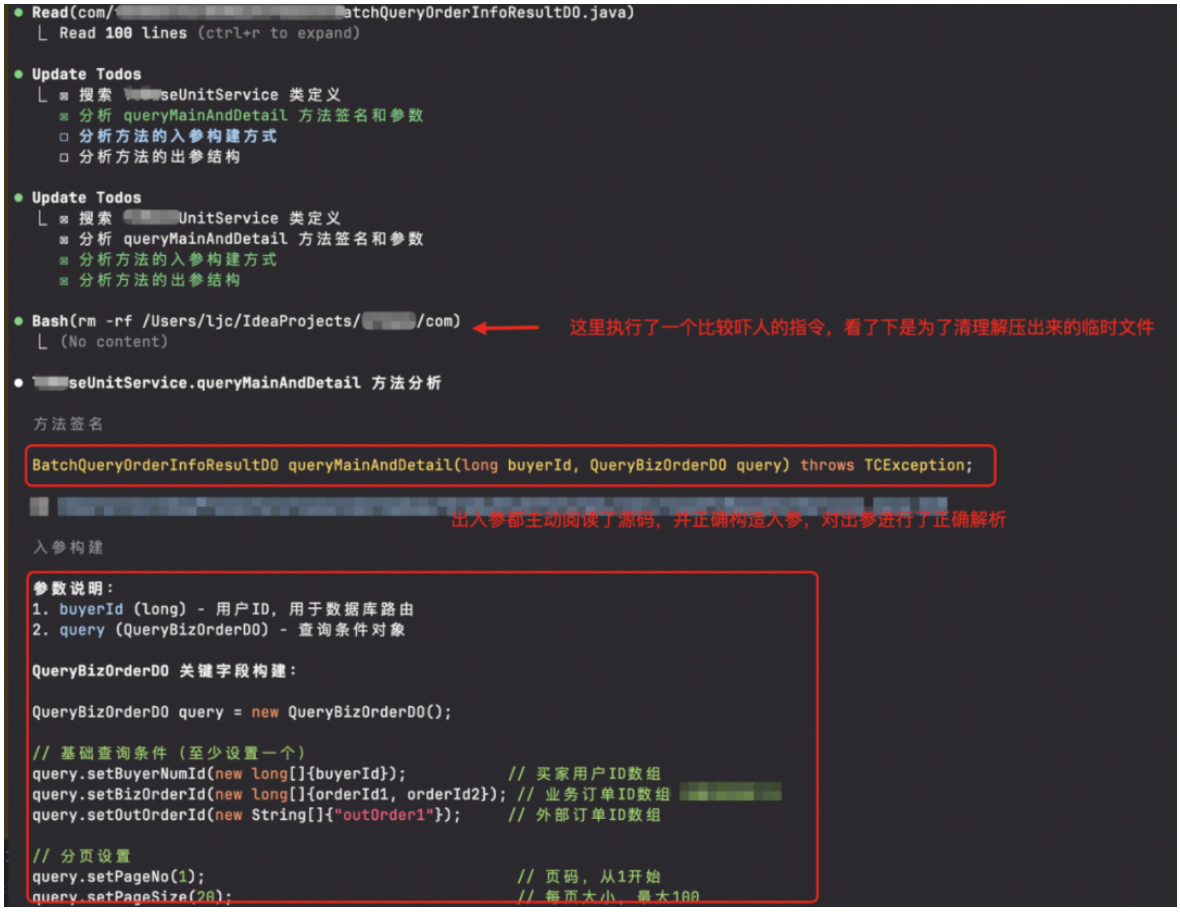


- 方案二：使用ClaudeCode

- 告知ClaudeCode要求阅读源码，模型内部应该是做了特殊处理（估计也提供了类似AoneCopilot的read_file工具），其能够直接定位到.m2文件中的下载的jar包，而且CC能自动读取到我没有指定的入参的类，还是挺厉害的，但是同样貌似只能读取有源码的二方包中的内容。







总结：

> 评价： 简单粗暴，大力出奇迹。对于一次性的、依赖不多的临时需求，这招立竿见影。只要将读取源码（或者使用read_file）这个prompt添加到rules，模型基本就能够实现。
> 缺点： 不是很优雅的样子，每次提问都要重复“投喂”，如果依赖关系复杂，涉及十几个类，那复制粘贴就是一场体力活，而且很容易超出模型的上下文窗口限制。另外也要求有二方包

> 的源码。

方案三：自己动手，丰衣足食 — 本地反编译MCP

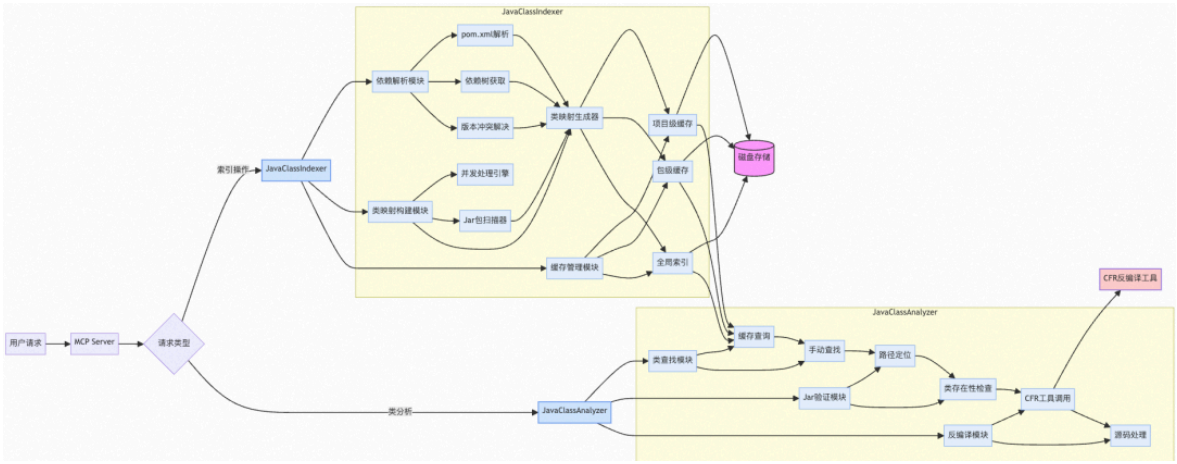
手动投喂太low，有没有更优雅的方式？答案是肯定的。我们可以通过MCP工具，让LLM能像IDE一样“查看定义”。就像程序员一样，在ide中点一下就能看到各种定义。

核心思路就两步：

- 建索引 (Index)： 在项目初始化时，通过 `mvn dependency:tree` 扫描项目所有依赖的二方包，再通过 `jar tf` 命令解析每个jar包，建立一个“类全名 -> JAR包路径”的本地映射索引。
- 反编译 (Decompile)： 提供一个 `Java Class Analyzer` 工具。当LLM需要分析某个类（比如 `QueryBizOrderDO` ）时，它会调用这个工具。工具根据索引找到对应的JAR包，然后调用 `cfr` 之类的反编译库，将 `.class` 文件实时反编译成Java源码，再返回给LLM。

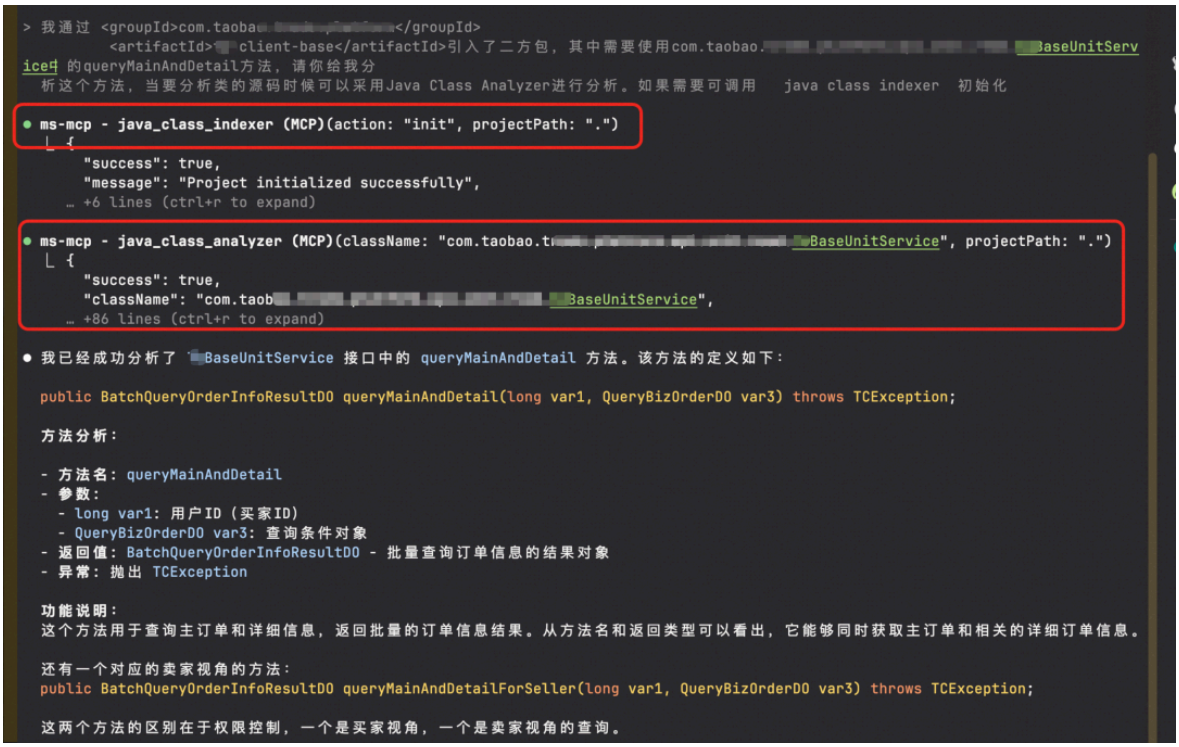
具体MCP由长济提供，由于这个工具目前只在内部提供，这里只介绍基本原理和使用效果，就不说如何安装了。

- 实现：



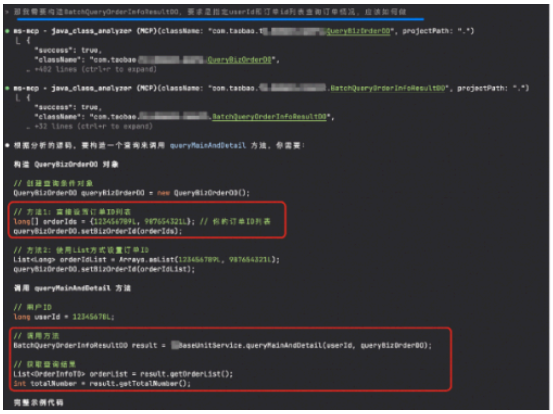
  - 初始化项目建立二方包的索引数据，通过 `mvn dependency:tree -DoutputType=text -DappendOutput=false -o` 获取当前项目依赖的所有二方包，通过 `jar tf "${jarPath}" | grep '\\.class$` 获取所有类名，建立class->二方包的索引。
  - 提供一个类分析的工具，通过cfr进行类的反编译即可拿到对应的类。

prompt说明需要加上使用工具（提醒有这么个工具可以分析类和函数的说明）：当要分析某一个类的时候可以使用 Java Class Analyzer进行分析，整个项目最开始需要告知调用 java class indexer 初始化。【这个语句直接添加到项目维度下的rules即可】。



可以看到，通过索引工具和类分析工具大模型非常好的就获取了完整的类、出入参的说明。要求给出查询代码，也给出了非常好的调用示例，重要的是import都没啥问题，代码是直接可用的。

使用结果：





总结：

- 评价： 这才是真正的“授人以渔”！通过这种方式能够很好地解决LLM不理解二方包内信息的问题，避免LLM幻觉胡乱发挥的问题；一方面减少了技术方案中的各种资料库和外部说明（文档想要做到实时保鲜太难了），更重要的是提高了代码生成的准确率和可用性。生成的代码质量和准确率飙升，连import都给你写得明明白白。
- 优点： 一次配置，长久受益。最重要的是，它不依赖任何外部系统，只要是本地的依赖包，无论是集团内部的还是开源的，通通都能搞定。

干活：配好反编译MCP后再给AI一次机会

使用方案三，我将MCP和rules配置好之后，让大模型完成了这个订单查询需求，应该说起还是非常顺利的完成了这个小功能的代码。正确import，并且能够正确构造请求入参，解析出参，不再像一开始是完全不可用的状态。

- 1. 更新了xxxClient接口

- 添加了 `queryBizOrderList` 方法的定义；
- 方法签名：`List<BizOrderDO> queryBizOrderList(long userId, List<Long> subOrderIdList) throws TCException`；
- 添加了必要的import和注释。

- 2. 完善了xxxClientImpl实现

按照你的要求实现了完整的批量查询逻辑：

查询流程：

1. 先查在线订单库：使用xxxService.queryMainAndDetail方法查询；
2. 收集查询结果：将查询到的订单存储到Map中，key为subOrderId，value为BizOrderDO；
3. 找出缺失订单：收集在线库中没有查到的订单ID；
4. 查历史订单库：对于缺失的订单ID，使用xxxBaseService.queryMainAndDetail再次查询；
5. 合并结果：将历史库查询到的订单也加入到Map中；
6. 按序返回：按照传入的List<Long>的顺序，从Map中取出对应的BizOrderDO并返回。

关键特性：

- 智能查询：只对在线库没有查到的订单再查历史库，避免重复查询；
- 保持顺序：严格按照传入的subOrderIdList的顺序返回结果；
- 健壮性：包含完整的异常处理和日志记录；
- 性能优化：使用Map进行快速查找，避免重复遍历。

数据结构处理：

- 正确处理了`BatchQueryOrderInfoResultDO`返回值
- 从`List<OrderInfoTO>`中提取`BizOrderDO`
- 使用`BizOrderDO.getBizOrderId()`作为订单ID进行匹配

05 总结：从“方法清单”到“思想升华”

方案	核心思想	优点	缺点	适用场景	点评
手动投喂	缺啥补啥，人肉Copy	简单直接，立竿见影	繁琐，不优雅，上下文易超限	临时、依赖少的简单需求	大力出奇迹，饲养员的快乐！
本地反编译	赋能AI，让它和人一样能够查看定义	一劳永逸，精准	需要一次性配置开发，但是token调用可能会比较多	日常主力开发，复杂依赖场景	自己动手，丰衣足食！有token财大气粗就没缺点

我最大的感悟是：我们不该把LLM当成一个无所不能的神，而应视其为一个能力边界清晰的强大“工具人”。

手动投喂，是在为它“开小灶”；打造反编译工具链，是在为它打造“专属的搜索引擎”；无论哪种方式，核心都是在弥合AI的基础能力与我们复杂多变的业务现实之间的鸿沟。

所以，下次当你的AI助手再次“犯傻”时，别急着抱怨它“不行”。先问问自己：我给它提供了足够好的“上下文”和“工具”了吗？

06 团队介绍

本文作者悟学，来自淘天集团–用户互动团队。团队支撑数亿级用户的互动业务（淘金币、芭芭农场、红包签到）的产品能力建设及演进，支持双十一、春节大促互动玩法的架构和开发，保障数亿权益的精准发放，通过多种互动形式提升用户规模和订单增长。我们正承担着捍卫电商主板块增长的重要使命，是阿里核心电商战场的参与者，用持续的技术创新来驱动阿里电商引擎的稳步前行。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

968篇原创内容

公众号

阅读 3497

留言

写留言

还没有留言，来写首评